

Projet de Recherche Opérationnelle

Algorithmes de flot dans les réseaux

Oussama Belhout — M1 — UNICA

Année universitaire 2025–2026

Table des matières

1	Introduction	2
2	Structure du graphe résiduel	2
2.1	Choix de la représentation	2
2.2	Astuce du flot symétrique	2
2.3	Limitation	2
3	Flot maximum : Edmonds–Karp	2
3.1	Principe	2
3.2	Complexité	3
3.3	Coupe minimum	3
4	MCF par Bellman–Ford	3
4.1	Principe	3
4.2	Pré-requis : pas de cycle négatif	3
4.3	Pseudo-code	4
4.4	Complexité	4
5	MCF par Dijkstra + Johnson	4
5.1	Idée	4
5.2	Initialisation et mise à jour	4
5.3	Récupération du coût réel	4
5.4	Pseudo-code	5
5.5	Complexité	5
6	Détection de cycles négatifs	5
6.1	Méthode principale : Bellman–Ford	5
6.2	Extraction du cycle	5
6.3	Fallback DFS	5
7	Comparaison des deux approches MCF	6
8	Tests et résultats	6
9	Conclusion	6

1 Introduction

Ce projet implémente en Python pur (sans bibliothèque externe spécialisée) trois familles d’algorithmes de flot :

- flot maximum par **Ford–Fulkerson** (variante *Edmonds–Karp*, BFS) ;
- flot de coût minimum (MCF) par augmentations successives, en deux versions : **Bellman–Ford** et **Dijkstra + Johnson** ;
- détection de **cycles négatifs** (Bellman–Ford, fallback DFS).

Seules `collections` et `heapq` sont utilisées.

2 Structure du graphe résiduel

2.1 Choix de la représentation

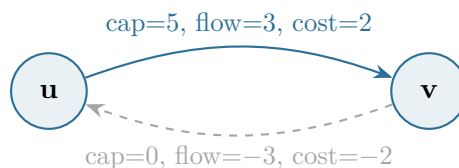
Deux options classiques :

- *matrice d’adjacence* : accès $O(1)$ mais $\Theta(V^2)$ en mémoire, parcours des voisins en $O(V)$;
- *dict de dict* : `graph[u][v] = {capacity, flow, cost}`, accès $O(1)$ amorti, mémoire $O(|E|)$, parcours en $O(\deg(u))$.

On a retenu le **dict de dict**, plus adapté aux graphes creux qu’on manipule en MCF.

2.2 Astuce du flot symétrique

Pour chaque arc réel (u, v) ajouté, on crée *aussi* une entrée `graph[v][u]` représentant l’arc inverse résiduel (capacité 0, coût $-w$) :



Le flot est stocké de manière *symétrique* (positif sur l’arc réel, négatif sur l’inverse) pour que la formule unique

$$\text{residual_capacity}(u, v) = \text{capacity}(u, v) - \text{flow}(u, v)$$

fonctionne dans les deux directions. Ainsi, augmenter de δ sur (u, v) se réduit à :

```
graph[u][v]['flow'] += delta
graph[v][u]['flow'] -= delta
```

Cette uniformité évite de manipuler deux structures séparées et rend le code des algorithmes beaucoup plus propre.

2.3 Limitation

Les arcs antiparallèles réels $((u, v)$ et (v, u) tous deux réels) ne sont pas supportés par cette structure — on lève une `ValueError` plutôt que d’accepter silencieusement un état incohérent. La solution standard consisterait à dédoubler l’un des deux arcs avec un sommet intermédiaire.

3 Flot maximum : Edmonds–Karp

3.1 Principe

Tant qu’il existe un chemin augmentant $s \rightarrow t$ dans le graphe résiduel, on pousse le bottleneck le long de ce chemin et on met à jour le résiduel. On choisit le chemin par **BFS** (plus court en

nombre d'arcs), ce qui garantit la borne polynomiale d'Edmonds–Karp.

Algorithme 1 Edmonds–Karp

```
1: function FORDFULKERSON( $G, s, t$ )
2:   maxflow  $\leftarrow 0$ 
3:   tant que vrai faire
4:      $P \leftarrow \text{BFS}(G_r, s, t)$ 
5:     si  $P = \emptyset$  alors sortir
6:     fin si
7:      $\delta \leftarrow \min\{c_r(u, v) : (u, v) \in P\}$ 
8:     Augmenter  $P$  de  $\delta$ 
9:     maxflow  $+= \delta$ 
10:  fin tant que
11:   $S \leftarrow$  sommets atteignables depuis  $s$  dans  $G_r$ 
12:   $T \leftarrow V \setminus S$ 
13:  renvoyer (maxflow,  $S$ ,  $T$ , coupe =  $\{(u, v) \text{ réel} : u \in S, v \in T\}$ )
14: fin function
```

3.2 Complexité

$$O(V \cdot E^2)$$

(BFS en $O(V + E)$, $O(VE)$ augmentations par le résultat d'Edmonds–Karp.)

3.3 Coupe minimum

Après convergence, un BFS depuis s sur le résiduel donne l'ensemble S des sommets atteignables. Les arcs réels (u, v) avec $u \in S$ et $v \notin S$ forment la coupe minimum (ils sont saturés). Par le théorème max-flow / min-cut, la capacité de cette coupe est égale à la valeur du flot maximum — propriété utilisée comme sanity check dans les tests.

4 MCF par Bellman–Ford

4.1 Principe

Analogie à Ford–Fulkerson, mais on choisit à chaque itération le chemin de *coût minimum* dans le résiduel (au lieu du plus court en nombre d'arcs). On utilise **Bellman–Ford** parce que le résiduel contient des arcs inverses de coût $-w$, donc Dijkstra ne s'applique pas directement.

4.2 Pré-requis : pas de cycle négatif

Si le graphe contient un cycle de coût strictement négatif et de capacité positive, le problème est *non borné* (on peut faire tourner du flot autour du cycle indéfiniment). On lance donc une détection préalable et on avorte avec un message clair en cas de cycle.

4.3 Pseudo-code

Algorithme 2 MCF par augmentations successives (Bellman–Ford)

```
1: function MCF-BF( $G, s, t, F_{\max}$ )
2:   si  $G$  contient un cycle négatif alors abort
3:   fin si
4:    $\text{cost}, \text{flow} \leftarrow 0, 0$ 
5:   tant que  $\text{flow} < F_{\max}$  faire
6:      $(d, \text{parent}) \leftarrow \text{BELLMAN-FORD}(G_r, s)$ 
7:     si  $d[t] = +\infty$  alors sortir
8:     fin si
9:      $P \leftarrow$  chemin reconstruit
10:     $\delta \leftarrow \min\{c_r(u, v) : (u, v) \in P\}$ 
11:     $\text{cost} += d[t] \cdot \delta, \quad \text{flow} += \delta$ 
12:    Augmenter  $P$  de  $\delta$ 
13:  fin tant que
14:  renvoyer  $(\text{cost}, \text{flow})$ 
15: fin function
```

4.4 Complexité

$$O(F \cdot V \cdot E)$$

où F est le volume de flot poussé. Lourd dès que F est grand, d'où la variante Dijkstra+Johnson.

5 MCF par Dijkstra + Johnson

5.1 Idée

On veut Dijkstra à chaque itération (rapide grâce à **heapq**) mais le résiduel contient des coûts négatifs. La solution de Johnson est de *reweighter* les coûts à l'aide de **potentiels** $h : V \rightarrow \mathbb{R}$:

$$w'(u, v) := w(u, v) + h[u] - h[v].$$

Deux propriétés essentielles de cette transformation :

1. **Préservation des plus courts chemins** : le long d'un chemin $s \rightarrow t$, les termes $h[u] - h[v]$ se télescopent et la somme totale ne change que d'une constante ($h[s] - h[t]$). Le chemin optimal pour w' est donc aussi optimal pour w .
2. **Positivité des coûts reweightés** : si h est choisi comme les distances depuis s dans le résiduel, alors $w'(u, v) \geq 0$ partout (conséquence de l'inégalité triangulaire des plus courtes distances).

5.2 Initialisation et mise à jour

- Initialement, h est calculé par **un seul** Bellman–Ford sur le graphe original.
- Après chaque Dijkstra, on met à jour $h[v] += d[v]$, ce qui maintient la positivité de w' pour l'itération suivante.

5.3 Récupération du coût réel

Comme Dijkstra travaille sur w' , on obtient le coût réel par la formule de télescopage :

$$\text{cost réel}(s \rightarrow t) = d_{w'}[t] + h[t] - h[s]$$

5.4 Pseudo-code

Algorithme 3 MCF par Dijkstra + reweighting de Johnson

```
1: function MCF-JOHNSON( $G, s, t, F_{\max}$ )
2:   si  $G$  contient un cycle négatif alors abort
3:   fin si
4:    $h \leftarrow \text{BELLMAN-FORD}(G, s)$  ▷ potentiels initiaux
5:    $\text{cost}, \text{flow} \leftarrow 0, 0$ 
6:   tant que  $\text{flow} < F_{\max}$  faire
7:      $(d, \text{parent}) \leftarrow \text{DIJKSTRA}(G_r, s, w')$ 
8:     si  $d[t] = +\infty$  alors sortir
9:     fin si
10:     $\text{cost\_unit} \leftarrow d[t] + h[t] - h[s]$ 
11:     $P \leftarrow$  chemin reconstruit
12:     $\delta \leftarrow \min\{c_r(u, v) : (u, v) \in P\}$ 
13:     $\text{cost} += \text{cost\_unit} \cdot \delta, \quad \text{flow} += \delta$ 
14:    Augmenter  $P$  de  $\delta$ 
15:     $h[v] += d[v]$  pour tout  $v$  atteignable
16:   fin tant que
17:   renvoyer ( $\text{cost}, \text{flow}$ )
18: fin function
```

5.5 Complexité

$$O(V \cdot E + F \cdot (V + E) \log V)$$

Strictement meilleur que la version Bellman–Ford pure.

6 Détection de cycles négatifs

6.1 Méthode principale : Bellman–Ford

Après $|V| - 1$ itérations de relaxation, Bellman–Ford a stabilisé les distances *si et seulement si* il n’y a pas de cycle négatif accessible. Une $|V|$ -ième itération qui améliore encore une distance signale donc l’existence d’un cycle négatif.

Pour détecter les cycles non accessibles depuis une source donnée, on initialise $\text{dist}[v] = 0$ pour tout v (équivalent à une source virtuelle reliée à tous les sommets).

6.2 Extraction du cycle

Quand un sommet v se fait relaxer à la $|V|$ -ième itération, on remonte $|V|$ fois via le tableau **parent** (garantissant qu’on aboutit dans le cycle), puis on suit **parent** jusqu’à retomber sur le sommet de départ pour reconstituer le cycle.

6.3 Fallback DFS

On a aussi une variante DFS qui trouve un cycle quelconque (via détection d’arête arrière) puis vérifie son poids. Plus limitée que Bellman–Ford mais utile comme second avis sur les petites instances.

7 Comparaison des deux approches MCF

Critère	c		
	MCF Bellman–Ford	MCF Dijkstra + Johnson	
Complexité / itération	$O(V \cdot E)$	$O((V + E) \log V)$	
Complexité totale	$O(F \cdot V \cdot E)$	$O(VE + F(V + E) \log V)$	
Coûts négatifs	Directement	Après reweighting initial	
Difficulté du code	Faible	Moyenne	

En pratique : sur nos jeux de tests (4 à 10 sommets), les deux algorithmes finissent en quelques millisecondes ; la différence devient visible sur les instances plus grosses ou à grandes capacités. La **cross-validation** entre les deux (test `test_mcf_dijkstra_johnson_matches_bf`) est la meilleure *sanity check* de l'implémentation : on vérifie qu'ils renvoient exactement le même coût et le même flot.

8 Tests et résultats

Le fichier `tests/test_all.py` contient 11 tests couvrant :

- flot max + conservation du flot + théorème max-flow / min-cut ;
- MCF Bellman–Ford avec arc de coût négatif ;
- MCF Dijkstra+Johnson, comparé à MCF-BF sur les mêmes graphes ;
- détection de cycle négatif (vrai positif et vrai négatif) ;
- avortement propre de MCF face à un cycle négatif ;
- stress test sur 10 sommets.

Tous les tests passent (11/11). L'exécution de `main.py` produit les sorties demandées dans le sujet.

9 Conclusion

Trois enseignements principaux :

1. Le choix de la structure de données est central : l'astuce du **flot symétrique** (flot négatif sur l'arc inverse) unifie le traitement des deux directions et simplifie tout le code en aval.
2. La **renormalisation de Johnson** permet de remplacer Bellman–Ford par Dijkstra dans le MCF, avec un gain de complexité significatif sur les instances de taille réaliste.
3. La **cross-validation** de deux implémentations indépendantes (Bellman–Ford vs Dijkstra+Johnson) reste l'outil de validation le plus convaincant.